

# Practice Exam Platform

## Architecture Flows

Flow 1: AI Study Assistant (modal to CachedChatResponse)

Flow 2: Exam Questions (ExamViewSet to QuestionGenerator)

*aws-project-4f082 | Web + Django backend*

## Flow 1: AI Study Assistant

Traces the path from the web UI modal through the Django backend to SyllabusAssistantService and CachedChatResponse. Both web and mobile share the same Firebase project (aws-project-4f082).

### User journey (step by step)

- User opens AI Study Assistant modal and picks a syllabus (solutions\_architect, cloud\_practitioner, or developer).
- AIAssistantModal calls onSelectSyllabus, then App.tsx handleSyllabusSelection.
- App calls api.ts getSyllabusLectures(syllabus) -> POST /assistant/syllabus-lectures/.
- urls.py syllabus\_assistant checks CachedLecturePlan (7-day TTL).
- On cache miss: SyllabusAssistantService.generate\_lecture\_plan() via Groq/DeepSeek.
- Lecture plan saved to CachedLecturePlan; UI shows roadmap + chat panel.
- User sends a message via sendChatMessage and streamChatWithSyllabusAssistant.
- POST /assistant/chat/stream/ (Server-Sent Events).
- Standalone questions only (history length <= 1): lookup CachedChatResponse by hash.
- Cache hit: instant SSE response, provider=cache. Miss: stream from AI, save to DB.
- App.tsx onDelta updates chat bubble with typing effect; onDone handles off-topic.

### Key files

```
typescript_simplified_app_with_timer/src/components/AIAssistantModal.tsx
typescript_simplified_app_with_timer/src/App.tsx (handleSyllabusSelection, sendChatMessage)
typescript_simplified_app_with_timer/src/utils/api.ts (getSyllabusLectures, streamChatWithSyllabusAssistant)
backend/exams/urls.py (syllabus_assistant, syllabus_assistant_chat_stream)
backend/exams/assistant.py (SyllabusAssistantService)
backend/exams/models.py (CachedLecturePlan, CachedChatResponse)
```

### Assistant cache rules

- CachedLecturePlan: one row per syllabus, 7-day staleness check.
- CachedChatResponse: global cache keyed by (syllabus, MD5 of normalized question).
- Only standalone questions cached (history <= 1); multi-turn bypasses cache.
- Off-topic messages filtered by regex before AI call; UI shows amber warning.

### Sequence (text diagram)

```
User -> AIAssistantModal -> App.tsx -> api.ts
      -> POST /assistant/syllabus-lectures/ -> CachedLecturePlan | SyllabusAssistantService
User -> chat input -> streamChatWithSyllabusAssistant
      -> POST /assistant/chat/stream/ -> CachedChatResponse | stream_chat_about_syllabus
      -> SSE deltas -> App onDelta -> modal chat bubble
```

## Flow 2: Exam Questions

Traces how a user starts a practice exam: pre-generation fills the question pool via QuestionGenerator; random\_questions serves a sample from PostgreSQL with a Django cache layer for question IDs.

### User journey (step by step)

- User clicks an exam tab (e.g. Solutions Architect).
- App.tsx handleExamTypeChange calls preGenerateExamQuestions(exam\_type).
- POST /exams/pre-generate/ hits ExamViewSet.pre\_generate\_questions.
- DB-first: if questions exist, return immediately (source: database).
- If pool empty: QuestionGenerator (Manus then OpenAI fallback) creates questions.
- Questions + answers saved to PostgreSQL; invalidate\_exam\_cache(exam\_id).
- Exam page loads getOrGenerateExamQuestions (reads DB only, no AI).
- getExamsByType then GET /exams/{id}/random-questions/?limit=50.
- ExamViewSet.random\_questions: Django cache exam\_{id}\_question\_ids (1 hour).
- Random sample up to 100 from pool, return 50 with prefetched answers.
- App transforms API format and starts timed exam UI.

### Key files

```
typescript_simplified_app_with_timer/src/App.tsx (handleExamTypeChange, loadQuestions)
typescript_simplified_app_with_timer/src/utils/api.ts (preGenerateExamQuestions, getOrGenerateExamQuestions)
backend/exams/views.py (pre_generate_questions, random_questions, invalidate_exam_cache)
backend/exams/services.py (QuestionGenerator)
backend/exams/models.py (Exam, Question, Answer)
```

### Exam cache rules

- Primary store: Question and Answer rows in PostgreSQL (permanent).
- Hot cache: Django cache key exam\_{id}\_question\_ids, 1-hour TTL.
- AI runs only when DB pool is empty (or force\_generate=true).
- invalidate\_exam\_cache() clears ID list after new questions are generated.

### Sequence (text diagram)

```
User -> exam tab -> preGenerateExamQuestions -> POST /exams/pre-generate/
      -> DB count | QuestionGenerator -> save Question/Answer -> invalidate cache
User -> exam page -> getOrGenerateExamQuestions -> getRandomExamQuestions
      -> GET /exams/{id}/random-questions/ -> cache IDs | DB sample -> 50 questions
```

## Side-by-side comparison

| Aspect         | AI Assistant                          | Exam Questions                 |
|----------------|---------------------------------------|--------------------------------|
| Primary store  | CachedLecturePlan, CachedChatResponse | Question, Answer tables        |
| Hot cache      | None (DB per request)                 | Django cache (question IDs)    |
| AI provider    | Groq then DeepSeek                    | Manus then OpenAI              |
| When AI runs   | Lecture/chat cache miss               | Only when DB pool empty        |
| Reuse scope    | Global (shared chat answers)          | Per-exam pool (all users)      |
| Invalidation   | 7-day staleness                       | invalidate_exam_cache + 1h TTL |
| Frontend entry | AIAssistantModal + stream chat        | Exam tab + random-questions    |
| Routing hub    | urls.py (assistant views)             | views.py (ExamViewSet)         |

### Shared pattern

Both systems follow serve-from-store-first, call-AI-only-on-miss. The assistant caches AI text responses for instant replay. The exam system caches structured question entities and samples them randomly. Same philosophy, different granularity.

### Product connection

Both flows use the same syllabus keys (solutions\_architect, cloud\_practitioner, developer) and the same api.ts client. A user can study with the AI tutor (Flow 1), then switch to the exam tab (Flow 2) for the same certification track.

### Why SyllabusAssistantService bridges communities (graph insight)

SyllabusAssistantService does not call views.py directly. urls.py is the hub that wires assistant endpoints and ExamViewSet into one API surface. Both share models.py and the same certification domain. The knowledge graph flagged high betweenness because this service sits at the intersection of AI study and exam practice.